

Synopsys.ai Copilot Administrator Guide

Version Y-2026.03, March 2026



Contents

Conventions	5
Customer Support	6
Statement on Inclusivity and Diversity	6
1. Introduction to the Synopsys.ai Copilot Knowledge Assistant	8
2. Installing Synopsys.ai Copilot	9
Installing the Synopsys.ai Copilot Software	9
Installing the SCS On-Premises Inference Server (Optional)	10
Initializing the PostgreSQL Database Server	10
3. Configuring the Endpoints	13
Model Endpoint Configurations	13
Embedding Model Configurations	13
Rerank Model Configurations	14
LLM Model Configurations	15
Database Endpoint Configurations	16
Using an Endpoint Configuration File	17
YAML Model Configurations	17
YAML Database Configurations	18
Referencing Environment Variables in Endpoint Configurations	19
Using SCS AI Gateway Autoconfiguration	19
4. Configuring the Application	21
Referencing Environment Variables in Application Settings	22
Updating Knowledge Assistant Configuration Settings at Runtime	22
Application Configuration Options	23
5. Configuring Logging Behavior	25
Introduction to System and Feedback Logging	25

Configuring Knowledge Assistant Logging Directory	26
Configuring System and Feedback Logging	26
Creating Custom Feedback Logging Fields	30

6. The copilot_admin Administration Utility	32
Introduction	32
Database Commands (db)	32
db show	32
db init	33
db test	34
db destroy	35
Content Commands (docs)	35
docs list	36
docs dump	36
docs restore	37
docs delete	37
Log Commands (logs)	38
logs dump	38
logs import_json	39
logs upgrade	40
logs downgrade	40
Model Commands (model)	41
model show	41
model test	42
model ask	43
model list	43

A. Host-Tool-Based Invocation Information	45
--	-----------

B. Supported Models	46
Supported Embedding Models	46
Supported Rerank Models	46
Supported LLM Models	47

Copyright and Proprietary Information Notice

© 2025 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

About This Document

The Synopsys.ai Copilot Knowledge Assistant Administrator guide provides information on administrative tasks, such as installation and configuration.

This preface includes the following sections:

- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
<code>Courier</code>	Indicates syntax, such as <code>write_file</code>
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <pre>prompt> write_file top</pre>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.

Convention	Description
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable

to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to the Synopsys.ai Copilot Knowledge Assistant

The Synopsys.ai Copilot Knowledge Assistant is an AI-powered assistant that uses a large language model (LLM) to understand your natural-language questions and generate natural-language responses. Knowledge Assistant considers multiple information sources, such as product documentation, man pages, knowledge articles, glossaries, and acronym definitions.

Knowledge Assistant is used together with a Synopsys host tool that supports it, such as the PrimeTime and Fusion Compiler tools. It can be invoked either from within a host tool session or directly from a Linux shell. See [Host-Tool-Based Invocation Information](#) for details.

Knowledge Assistant requires a GenAI_KA_Copilot or GenAI_KA_Copilot_Beta license.

The administrative tasks detailed in this guide are as follows:

- [Installing Synopsys.ai Copilot](#)
- [Configuring the Endpoints](#)
- [Configuring the Application](#)
- [Configuring Logging Behavior](#)

Note:

For illustration purposes, this document uses screenshots and procedures for the Fusion Compiler Knowledge Assistant. The screenshots and procedures are applicable to other supported Synopsys products.

2

Installing Synopsys.ai Copilot

The following topics describe how to install Synopsys.ai Copilot and its associated services:

- [Installing the Synopsys.ai Copilot Software](#)
- [Installing the SCS On-Premises Inference Server \(Optional\)](#)
- [Initializing the PostgreSQL Database Server](#)

Installing the Synopsys.ai Copilot Software

Prerequisites:

- Licenses for the desired host tools must be available. See [Host-Tool-Based Invocation Information](#) for details.
- You must have access to the Synopsys Installer.

A single Synopsys.ai Copilot installation is used for all supported host tools.

To install the Synopsys.ai Copilot Knowledge Assistant,

1. Download Knowledge Assistant Synopsys Product File (.spf) using the provided EST download instructions.
 - `copilot_INSTALL_README.txt`
 - `copilot_vY-2026.03_linux64.spf`
 - `copilot_vY-2026.03_common.spf`
2. Using the Synopsys Installer, install Knowledge Assistant .spf to the required installation directory.

For information about downloading and running the Synopsys Installer, go to the Synopsys Installation Guide page at <https://www.synopsys.com/install>, where you can generate the Installing Synopsys® Tools document.

Note:

The Synopsys.ai Copilot Knowledge Assistant must be installed in its own directory, not into an existing Synopsys tool installation directory.

Installing the SCS On-Premises Inference Server (Optional)

Synopsys.ai Copilot uses services hosted by the Synopsys Cognitive Services (SCS) platform. This platform is used to efficiently host and share GPU and database resources across all Synopsys-developed AI applications.

The *Synopsys Cognitive Services (SCS) Deployment Guide* provides detailed instructions for deploying Synopsys AI applications on this platform. It covers on-premises LLM deployment (supporting Docker Compose, Kubernetes, air-gapped environments, and fine-tuned models) and cloud-based LLM deployment. The guide also includes setup instructions for vector databases, inference servers, and a centralized LLM gateway to manage requests to multiple LLM providers, ensuring consistent configuration across Synopsys GenAI products.

Initializing the PostgreSQL Database Server

Synopsys.ai Copilot uses the [PostgreSQL database](#) (often known simply as “Postgres”) to store and retrieve data.

The requirements for the Postgres server are as follows:

- PostgreSQL version 16 or later is required.
- The database server must have the [pgvector](#) extension installed.
- The database server must be network-accessible from the machines where Copilot is used.

You can install and run a dedicated Postgres server, or you can use an existing Postgres server (if it meets the requirements above).

Copilot uses two databases in the Postgres server, each for a different purpose:

Type of data	Default database name
Vector store documents	<code>copilot_docs</code>

Type of data	Default database name
User conversation history	<code>copilot_logs</code>

Copilot creates and uses multiple Postgres login accounts (also known as *roles*) to communicate with the Postgres server:

Postgres role	Purpose	Credentials provided by
Superuser	Create the <code>copilot_admin</code> and <code>copilot_user</code> roles during initial setup	<code>PGUSER</code> and <code>PGPASSWORD</code> environment variables
<code>copilot_admin</code>	Create the <code>copilot_docs</code> and <code>copilot_logs</code> databases Add, update, delete vector store content	<code>COPILOT_ADMIN_PGUSER</code> and <code>COPILOT_ADMIN_PGPASSWORD</code> environment variables
<code>copilot_user</code>	Allow end users to access the databases	Endpoint configuration file

To configure a Postgres server for use with Knowledge Assistant,

1. On the Postgres server host machine, compile and install the pgvector extension.

Installation instructions are available [here](#).

Note:

When using an SCS-provided Postgres container, this step is not needed.

2. Set the following environment variables to specify the Postgres superuser login information to use:

```
# Postgres superuser login information
export PGUSER=superuser_name
export PGPASSWORD='superuser_password'
```

Note:

When using an SCS Postgres container, the superuser role name and password were determined when the container was initially configured and deployed.

3. Set the following environment variables to specify the Copilot administrator role to create:

```
# Copilot administrator role
export COPILOT_ADMIN_PGUSER=copilot_admin_name
export COPILOT_ADMIN_PGPASSWORD='copilot_admin_password'
```

Important:

You must use different role names for the superuser, administrator, and user roles.

4. To verify that the required database configuration information is available in the endpoint configuration file, run the following command:

```
$SNPSAI_COPILOT_HOME/admin/bin/copilot_admin db show
```

5. To initialize the roles and databases, run the following command:

```
$SNPSAI_COPILOT_HOME/admin/bin/copilot_admin db init
```

6. To test the newly created roles and databases, run the following command:

```
$SNPSAI_COPILOT_HOME/admin/bin/copilot_admin db test
```

7. To upload a set of vector store (*.sch) files, run the following command:

```
$SNPSAI_COPILOT_HOME/admin/bin/copilot_admin docs restore \  
tool1_*.sch tool2_*.sch
```

Multiple file names and wildcard patterns are supported.

To delete the databases and roles (such as to use a different configuration),

- Run the following command:

```
$SNPSAI_COPILOT_HOME/admin/bin/copilot_admin db destroy
```

The `copilot_docs` and `copilot_logs` databases can be hosted on the same Postgres server or on different servers.

Important:

If Synopsys.ai Copilot communicates with a Postgres server across a high-latency connection (such as between different geographical regions), there can be a significant increase in startup time and user interface response times.

3

Configuring the Endpoints

The following topics describe how to configure the model and database endpoints used by Synopsys.ai Copilot:

- [Model Endpoint Configurations](#)
- [Database Endpoint Configurations](#)
- [Using an Endpoint Configuration File](#)
- [Using SCS AI Gateway Autoconfiguration](#)

Model Endpoint Configurations

A *model configuration* is a data structure that describes the capabilities and access characteristics of a particular embedding model. Model endpoint configurations are organized into three categories: embedding, rerank, and LLM.

The following topics describe the configurations for each model type:

- [Embedding Model Configurations](#)
- [Rerank Model Configurations](#)
- [LLM Model Configurations](#)

Embedding Model Configurations

Each embedding configuration has a *configuration name* that is used to reference it. The configuration name must begin with `scs-gateway-v1-`.

An example configuration is as follows:

```
scs-gateway-v1-EMBEDDING-CONFIG-NAME :
  model_canonical_name: MODEL-CANONICAL-NAME
  model_route_name: MY-EMBEDDING-GATEWAY-ROUTE-NAME
  endpoint: https://SCS-LLM-GATEWAY.MYCOMPANY.COM/v1/embeddings
  api_key: 'MY-GATEWAY-API-KEY'
```

The following table lists the configuration settings used for embedding models.

Table 1: Settings for Embedding Models

Name	Type	Default	Description
<code>model_canonical_name</code>	str	none	Canonical (intrinsic) name of the model
<code>model_route_name</code>	str	none	Name of the model route or deployment
<code>endpoint</code>	str	none	Model endpoint URL to send the request to
<code>api_key</code>	str	none	API key to include for authentication
<code>dimensions</code>	int	none	Number of dimensions of the embedding vectors
<code>max_doc_tokens</code>	int	none	Maximum number of tokens per document
<code>max_input_tokens</code>	int	none	Maximum number of tokens across all documents
<code>max_input_bytes</code>	int	none	Maximum number of bytes across all documents
<code>max_doc_count</code>	int	1	Maximum number of documents per request
<code>max_requests</code>	int	1	Maximum number of concurrent requests
<code>max_retries</code>	int	3	Maximum number of retries
<code>timeout</code>	int	none	Timeout limit to wait for response, in seconds
<code>headers_kwargs</code>	dict	none	Additional fields to include in request headers

Rerank Model Configurations

Each rerank configuration has a *configuration name* that is used to reference it. The configuration name must begin with `scs-gateway-v1-`.

An example configuration is as follows:

```
scs-gateway-v1-RERANK-CONFIG-NAME:
  model_canonical_name: MODEL-CANONICAL-NAME
  model_route_name: MY-RERANK-GATEWAY-ROUTE-NAME
  endpoint: https://SCS-LLM-GATEWAY.MYCOMPANY.COM/v1/rerank
  api_key: 'MY-GATEWAY-API-KEY'
```

The following table lists the configuration settings used for rerank models.

Table 2: Settings for Rerank Models

Name	Type	Default	Description
<code>model_canonical_name</code>	str	none	Canonical (intrinsic) name of the model

Table 2: Settings for Rerank Models (Continued)

Name	Type	Default	Description
<code>model_route_name</code>	str	none	Name of the model route or deployment
<code>endpoint</code>	str	none	Model endpoint URL to send the request to
<code>api_key</code>	str	none	API key to include for authentication
<code>max_query_tokens</code>	in	none	Maximum number of tokens for query
<code>max_doc_tokens</code>	int	none	Maximum number of tokens per document
<code>max_querydoc_tokens</code>	in	none	Maximum number of tokens for query+documents
<code>max_input_tokens</code>	int	none	Maximum number of tokens across all documents
<code>max_doc_count</code>	int	none	Maximum number of documents per request
<code>max_retries</code>	int	3	Maximum number of retries
<code>timeout</code>	int	none	Timeout limit to wait for response, in seconds
<code>headers_kwargs</code>	dict	none	Additional fields to include in request headers

LLM Model Configurations

Each LLM configuration has a *configuration name* that is used to reference it. The configuration name must begin with `scs-gateway-v1-`.

An example configuration is as follows:

```
scs-gateway-v1-LLM-CONFIG-NAME:
  model_canonical_name: MODEL-CANONICAL-NAME
  model_route_name: MY-LLM-GATEWAY-ROUTE-NAME
  endpoint: https://SCS-LLM-GATEWAY.MYCOMPANY.COM/v1
  api_key: 'MY-GATEWAY-API-KEY'
```

The following table lists the configuration settings used for LLM models.

Table 3: Settings for LLM Models

Name	Type	Default	Description
<code>model_canonical_name</code>	str	none	Canonical (intrinsic) name of the model
<code>model_route_name</code>	str	none	Name of the model route or deployment
<code>endpoint</code>	str	none	Model endpoint URL to send the request to

Table 3: Settings for LLM Models (Continued)

Name	Type	Default	Description
<code>api_key</code>	str	none	API key to include for authentication
<code>temperature</code>	float	0	Temperature (variability) setting for generation
<code>max_input_tokens</code>	int	96000	Maximum number of input tokens to the model
<code>max_completion_tokens</code>	int	8000	Maximum number of completion tokens from the model
<code>max_total_tokens</code>	int	128000	Maximum number of input+completion tokens
<code>max_retries</code>	int	3	Maximum number of retries
<code>timeout</code>	int	none	Timeout limit to wait for response, in seconds
<code>headers_kwargs</code>	dict	none	Additional fields to include in request headers

Database Endpoint Configurations

A *database configuration* is a data structure that describes the capabilities and access characteristics of a particular database server.

Each database configuration has a *configuration name* that is used to reference it. Example configurations for `copilot_docs` and `copilot_logs` databases are as follows:

```
pg_docstore:
  hostname: SCS-POSTGRES.MYCOMPANY.COM
  port: 5432
  username: copilot_user
  password: 'COPILOT-USER-PASSWORD'
  database: copilot_docs

pg_logstore:
  hostname: SCS-POSTGRES.MYCOMPANY.COM
  port: 5432
  username: copilot_user
  password: 'COPILOT-USER-PASSWORD'
  database: copilot_logs
```

For more information about these database types, see [Initializing the PostgreSQL Database Server](#).

The following table lists the configuration settings used for database endpoints.

Table 4: Settings for Database Endpoints

Name	Data type	Description
hostname	str	URL of the Postgres database server or cluster
port	int	Port number of the Postgres database server or cluster
username	str	User name of the <code>copilot_user</code> role
password	str	Password of the <code>copilot_user</code> role
database	str	Name of the Postgres database

Using an Endpoint Configuration File

You can provide model and database configurations by specifying the location of a *endpoint configuration file*.

The endpoint configuration file is written in YAML format. By default, Synopsys.ai Copilot uses the endpoint configuration file at the following path:

- `$SNPSAI_COPILOT_HOME/admin/setup/generic_tool.yaml`

However, the location can be configured as described in [Configuring the Application](#), such as to share a common endpoint configuration file across multiple Copilot release installations.

YAML Model Configurations

The endpoint configuration file can include `embedding_config`, `rerank_config`, `llm_config` sections that contain their respective model configurations. For example,

```
# Global settings
rerank: true

# Embedding model configurations
embedding_config:
  my_embedding_config_name:
    model_canonical_name: ...
    endpoint: ...
    api_key: ...

# Rerank model configurations
rerank_config:
  my_rerank_config_name:
    model_canonical_name: ...
    endpoint: ...
```

```
    api_key: ...

# LLM configurations
llm_config:
    my_llm_config_name:
        model_canonical_name: ...
        endpoint: ...
        api_key: ...
```

If a single model configuration is defined in a given model section, it is automatically used. If multiple configurations are available, use the following additional top-level settings in the file to specify which model to use:

```
# global settings
...

embedding_model_name: my_embedding_config_name
rerank_model_name: my_rerank_config_name
llm_name: my_llm_config_name
```

In model configuration definitions, sensitive fields can be encrypted so that end users cannot see their actual values. For details, see [model show](#).

YAML Database Configurations

The endpoint configuration file must include a `database_configs` section that contains database configurations. For example,

```
# Database configurations
database_configs:
    pg_docstore:
        hostname: SCS-POSTGRES.MYCOMPANY.COM
        port: 5432
        username: copilot_user
        password: 'COPILOT-USER-PASSWORD'
        database: copilot_docs

    pg_logstore:
        hostname: SCS-POSTGRES.MYCOMPANY.COM
        port: 5432
        username: copilot_user
        password: 'COPILOT-USER-PASSWORD'
        database: copilot_logs
```

In database configuration definitions, sensitive fields can be encrypted so that end users cannot see their actual values. For details, see [db show](#).

Referencing Environment Variables in Endpoint Configurations

You can reference environment variables in model and database configuration settings by enclosing their names in curly braces.

For example,

```
embedding_config:
  my_embedding_config_name:
    ...
    endpoint: "{SNPSAI_GATEWAY_URL}/v1/embeddings"
    api_key: "{SNPSAI_GATEWAY_KEY_COPILOT}"
```

Using SCS AI Gateway Autoconfiguration

Synopsys.ai Copilot can contact SCS AI Gateway to automatically obtain the available model configurations. This feature is known as *autoconfiguration*. When autoconfiguration is used, model configurations are not needed in the endpoint configuration file.

Note:

This feature requires SCS AI Gateway release 2026.02.1 or later.

To use this feature, set the following two environment variables before invoking Synopsys.ai Copilot:

```
# csh/tcsh
setenv SNPSAI_GATEWAY_URL 'https://SCS-GATEWAY.MYCOMPANY.COM'
setenv SNPSAI_GATEWAY_KEY_COPILOT 'PUT_COPILOT_PRODUCT_KEY_HERE'

# sh/bash/zsh
export SNPSAI_GATEWAY_URL='https://SCS-GATEWAY.MYCOMPANY.COM'
export SNPSAI_GATEWAY_KEY_COPILOT='PUT_COPILOT_PRODUCT_KEY_HERE'
```

During invocation, if the `SNPSAI_COPILOT_LOGLEVEL` environment variable is set to `INFO`, the following messages are issued:

```
[INFO] ... - Using autoconfigured endpoint configurations from
SCS AI Gateway.
```

Model configuration names are formed from the Gateway route names by prepending an `scs-gateway-v1-` prefix to them.

To report the available models, you can run the following commands:

```
# Shows a summary table for all models
$SNPSAI_COPILOT_HOME/admin/bin/copilot_admin model list

# Shows all available model configurations
$SNPSAI_COPILOT_HOME/admin/bin/copilot_admin model show --all
```

When autoconfiguration is enabled, model configurations are obtained from both SCS AI Gateway and the endpoint configuration file. If configurations with the same name occur, the configuration from the file takes precedence (to allow for local overrides).

The `SNPSAI_GATEWAY_KEY_COPILOT` environment variable can be encrypted, but the `SNPSAI_GATEWAY_URL` environment variable cannot.

SCS AI Gateway model parameters are mapped to their equivalent Synopsys.ai Copilot model settings as shown in the following table.

Table 5: Model Parameter Mapping From SCS AI Gateway to Copilot

SCS AI Gateway model parameter	Synopsys.ai Copilot model setting
<code>canonicalModelName</code>	<code>model_canonical_name</code>
<code>embeddingDimensions</code>	<code>dimensions</code>
<code>maxCompletionTokens</code>	<code>max_completion_tokens</code>
<code>maxDocumentCount</code>	<code>max_doc_count</code>
<code>maxDocumentTokens</code>	<code>max_doc_tokens</code>
<code>maxInputBytes</code>	<code>max_input_bytes</code>
<code>maxInputTokens</code>	<code>max_input_tokens</code>
<code>maxQueryTokens</code>	<code>max_query_tokens</code>
<code>maxTotalTokens</code>	<code>max_total_tokens</code>
<code>routeName</code>	<code>model_route_name</code>

4

Configuring the Application

Prerequisite:

- An updated endpoint configuration file (generic_tool.yaml)

Application configuration information is applied in the following order, with settings in later sources taking precedence over those in earlier sources:

1. The \$SNPSAI_COPILOT_HOME/admin/setup/.synopsys_copilot.yaml file
2. The ~/.synopsys_copilot.yaml file
3. The ./synopsys_copilot.yaml file
4. The SNPSAI_COPILOT_CONFIG_FILE environment variable (using the specified file)
5. The SNPSAI_COPILOT_CONFIG_YAML environment variable (parsed as a YAML fragment)

To configure Knowledge Assistant for site-wide use, perform the following steps:

1. Set the site configuration file location.

A generic_tool.yaml file is needed that configures site-specific settings for Knowledge Assistant (such as LLM endpoints and keys). In one of the configuration sources above, set the generic_config_path setting to the location of this file:

```
# .synopsys_copilot.yaml
generic_config_path: /global/path/to/my_generic_tool.yaml
```

2. (Optional) Set the logging directory location.

Set the copilot_log_dir setting to the location of the logging directory:

```
# .synopsys_copilot.yaml
copilot_log_dir: /global/path/to/copilot_stuff
```

The copilot_log_dir setting can be set to either a single path or a list of paths. For more information, see [Configuring Knowledge Assistant Logging Directory](#).

3. (Optional) To further customize Knowledge Assistant behavior, additional configuration options are available. For more information, see [Application Configuration Options](#).

Additional user-level configuration might be required, such as setting the `SNPSAI_COPILOT_HOME` and `DISPLAY` environment variables. For details, see the *Copilot Knowledge Assistant User Guide*.

Note:

The Web UI for Synopsys.ai Copilot requires OpenID Connect (OIDC) Single Sign-On (SSO) for authentication. For details on Web-UI OIDC/OAuth2 authentication support, refer to: `$SNPSAI_COPILOT_HOME/linux64/copilot/bin/README-oauth-deployment.md`.

Referencing Environment Variables in Application Settings

You can reference environment variables in application configuration settings by enclosing their names in curly braces.

For example,

```
# .synopsys_copilot.yaml
generic_config_path: "{CONFIGS}/{GROUP}/endpoint_config.yaml"
```

Updating Knowledge Assistant Configuration Settings at Runtime

Knowledge Assistant application configuration settings can be updated at runtime by setting `SNPSAI_COPILOT_CONFIG_*` environment variables. This allows a single installation to be used under multiple configurations.

Using YAML Fragments

The `SNPSAI_COPILOT_CONFIG_YAML` environment variable is parsed as YAML content, then applied incrementally with the precedence described in [Configuring the Application](#).

For example, the LLM endpoint configuration could be updated depending on a user's project or business unit:

```
# for csh/tcsh
setenv SNPSAI_COPILOT_CONFIG_YAML \
  "generic_config_path: /path/to/specific_endpoint_config"

# for sh/bash/zsh
export SNPSAI_COPILOT_CONFIG_YAML=\
  "generic_config_path: /path/to/specific_endpoint_config"
```

Or, the logging directory could be configured depending on similar external factors:

```
# for csh/tcsh
setenv SNPSAI_COPILOT_CONFIG_YAML \
  "copilot_log_dir: /path/to/copilot_log_dir"
```

```
# for sh/bash/zsh
export SNPSAI_COPILOT_CONFIG_YAML=\
"copilot_log_dir: /path/to/copilot_log_dir"
```

Multiple settings can be specified using embedded line feed characters:

```
# for csh/tcsh
setenv SNPSAI_COPILOT_CONFIG_YAML \
  "setting1: value1\nsetting2: value2\n..."

# for sh/bash/zsh
# ('$' is needed to process escaped special characters)
export SNPSAI_COPILOT_CONFIG_YAML=\
'$setting1: value1\nsetting2: value2\n...'
```

Using YAML Files

The `SNPSAI_COPILOT_CONFIG_FILE` environment variable is parsed as a YAML file path. The contents of the file are read, then applied incrementally with the precedence described in [Configuring the Application](#).

Application Configuration Options

The following table shows the options you can set in the `synopsys_copilot.yaml` file.

Option	Description	Default
<code>copilot_log_dir</code>	Directory path where system and feedback log files are written to. If multiple users share the log directory, ensure all of them have read and write permissions.	None
<code>disable_browser</code>	Boolean to disable clickable links in the document list.	<code>false</code>
<code>enable_routing</code>	Boolean to enable question classification and routing to Workflow Assistant generation engine	<code>true</code>
<code>generic_config_path</code>	File path where the customer-specific endpoint configuration file is loaded from. This contains the information needed to access the required model and database resources.	<code>\$SNPSAI_COPILOT_HOME/adm in/setup/generic_tool.yaml</code>

Option	Description	Default
<code>hallucination</code>	Boolean to enable hallucination detection.	<code>true</code>
<code>ignore_proxy</code>	Boolean to determine whether proxy settings are to be disabled when making HTTP requests to the cloud server (for cloud-hosted installations only).	<code>false</code>
<code>ka_max_completion_tokens</code>	Integer to specify the maximum number of tokens that can be generated by the LLM	2000
<code>ka_max_total_tokens</code>	Integer to specify the maximum number of tokens that can be sent to the LLM	16000
<code>localhost_check</code>	Keyword indicating how to perform localhost queries. Valid values are <code>socket</code> and <code>http</code> .	<code>socket</code>
<code>query_max_length</code>	Integer to specify the maximum number of bytes that can be submitted as a question	6000

5

Configuring Logging Behavior

Administrators can configure logging so that user feedback and system performance can be monitored and optionally communicated with Synopsys.

Logging is described in the following topics:

- [Introduction to System and Feedback Logging](#)
- [Configuring Knowledge Assistant Logging Directory](#)
- [Configuring System and Feedback Logging](#)
- [Creating Custom Feedback Logging Fields](#)

Introduction to System and Feedback Logging

User feedback is essential to improve the responses generated by Knowledge Assistant. Each response has a mechanism to provide feedback. Users can submit feedback by rating each answer with a thumbs-up or thumbs-down. Users can also provide a short explanation of their rating using the comment box that appears when they rate the answer.

Logging data is organized as follows:

- System logging
 - Entries written for every question that is asked
 - Useful for assessing overall system usage and performance
- Feedback logging
 - Entries written only when feedback is submitted for an answer
 - Useful for investigating user feedback and satisfaction

The following topics describe how to configure and customize logging in Knowledge Assistant:

Configuring Knowledge Assistant Logging Directory

The `copilot_log_dir` configuration setting specifies the location of Knowledge Assistant logging directory.

For simple environments, set the `copilot_log_dir` setting to the desired location:

```
# .synopsys_copilot.yaml
copilot_log_dir: /global/path/to/logdir
```

The `copilot_log_dir` specification is processed as follows:

- If the path does not end with a final `copilot_logs` directory name, it is appended.
- If the `copilot_logs` directory exists at the specified path and is writable, it is used.
- If the `copilot_logs` directory does not exist at the specified path but can be successfully created there, it is created and used.

Note:

If multiple users share the log directory, ensure all of them have read and write permissions.

For more complex environments, you can specify a list of paths, and the first valid path with an existing or creatable `copilot_logs` directory is used:

```
# .synopsys_copilot.yaml
copilot_log_dir:
- /global/path1/to/logdir
- /global/path2/to/logdir
- /global/path3/to/logdir
```

If no valid logging directory can be found or created, on-disk logging is disabled.

Configuring System and Feedback Logging

Each time a user asks a question, Knowledge Assistant writes a *system* logging entry. Each time the user submits thumbs-up or thumbs-down feedback, Knowledge Assistant writes a *feedback* logging entry.

A `logging_config.yaml` file controls what system and feedback logging data is written. Knowledge Assistant looks for this file in the following locations (highest precedence first):

- `$SNPSAI_COPILOT_HOME/linux64/copilot/bin/_internal/product_data/ka/{tool}/logging_config.yaml`
- `$SNPSAI_COPILOT_HOME/linux64/copilot/bin/_internal/product_data/ka/logging_config.yaml`

If a tool-specific logging configuration file is not found, then the default logging configuration file is used.

The default `logging_config.yaml` configuration is as follows:

```
# logging_config.yaml
copilot_log_config:
  system:
    filepath: system/{tool}/{id}
    fields:
      - feedback_rating
      - system_client
      - system_model_endpoint
      - system_model_type
      - system_response_time
      - system_timestamp
      - system_token_rx
      - system_token_tx
      - system_user_agent
      - system_user_name
      - tool
  feedback:
    filepath: feedback/{tool}/{id}
    fields:
      - feedback_comment
      - feedback_rating
      - feedback_reason
      - feedback_was_ka_faster
      - system_client
      - system_model_type
      - system_timestamp
      - system_user_agent
      - system_user_name
      - tool
```

Note:

When a system logging entry file is first created after a question is answered, the `feedback_rating` field is set to an empty string. When feedback for that answer is submitted (or resubmitted, after deselecting the original rating), both the system and feedback logging entry files are updated accordingly.

By default, the system and feedback logging files are written to the following paths, respectively:

- `{copilot_log_dir}/{assistant}/system/{tool}/{id}.json`
- `{copilot_log_dir}/{assistant}/feedback/{tool}/{id}.json`

where:

- `copilot_log_dir` is defined in the `.synopsys_copilot.yaml` file
- `{assistant}` is `ka` (for Knowledge Assistant) or `wa` (for Workflow Assistant)

The portion of the paths after the `copilot_log_dir/{assistant}` directory can be configured using the `filepath` configuration parameter, referencing any of the following fields: `system_user_name`, `tool`, `id`. The `id` value should be at the end (as the last value is used to construct the logging entry file name).

The available system and feedback logging fields in the fields lists are as follows:

Field Name	Description
<code>conversation_id</code>	A Universally Unique Identifier (UUID) value that remains persistent for questions within a conversation, then changes when the user clicks the Start a New Topic button
<code>feedback_comment</code>	Optional comment from user-submitted feedback
<code>feedback_rating</code>	Rating from user-submitted feedback. Value is <code>like</code> (thumbs-up) or <code>dislike</code> (thumbs-down)
<code>session_id</code>	A Universally Unique Identifier (UUID) value that remains persistent throughout the user's session
<code>system_client</code>	Type of user Interface client used to access Knowledge Assistant
<code>system_first_token_client</code>	Timestamp when first token was received from LLM
<code>system_model_endpoint</code>	LLM model endpoint used to compute generative response
<code>system_model_type</code>	Name of LLM model used to compute generative response
<code>system_response_time</code>	Time from query to response (seconds)
<code>system_timestamp</code>	Timestamp when query was processed
<code>system_token_rx</code>	Number of tokens received from LLM

Field Name	Description
<code>system_token_tx</code>	Number of tokens sent to LLM
<code>system_user_agent</code>	Type and version of Copilot client used for query
<code>system_user_name</code>	User name
<code>tool</code>	Keyword indicating the tool for which Knowledge Assistant was used

Fields are automatically placed into the logging entry Q&A data structure where appropriate. For example, `tool` is placed in the root of the Q&A structure, while `feedback_rating` is placed into the metadata section of the Q&A structure.

Example Logging Entries

Each logging entry is written as a JSON file, named after the ID value associated with that Q&A.

The following examples show the system and feedback logging entries captured in the `system/` and `feedback/` directories for a single Q&A user interaction with feedback submission.

System logging entry file: `{copilot_log_dir}/ka/system/pt/c09fcb0f-b438-5067-9140-c50af6ddcab5.json`

```
{
  "id": "c09fcb0f-b438-5067-9140-c50af6ddcab5",
  "metadata": {
    "feedback_rating": "like",
    "system_client": "...",
    "system_first_token_timestamp": 1708078090
    "system_model_endpoint": "https://...",
    "system_model_type": "openai",
    "system_response_time": 3.2,
    "system_timestamp": 1708078090,
    "system_tokens_tx": "...",
    "system_tokens_rx": "...",
    "system_user_agent": "CopilotDesktop/1.0 (Y-2026.03)",
    "system_user_name": "someuser",
    "tool": "pt"
  }
}
```

Feedback logging entry file: `{copilot_log_dir}/ka/feedback/pt/c09fcb0f-b438-5067-9140-c50af6ddcab5.json`

```
{
  "id": "c09fcb0f-b438-5067-9140-c50af6ddcab5",
```

```
"tool": "pt",
"question": [
  "What color is the sky?"
],
"helper_answer": [
  "The sky is blue."
],
"metadata": {
  "feedback_comment": "Good answer!",
  "feedback_rating": "like",
  "feedback_reason": null,
  "feedback_was_ka_faster": null,
  "system_client": "...",
  "system_model_type": "openai",
  "system_user_agent": "CopilotDesktop/1.0 (Y-2026.03)",
  "system_user_name": "someuser",
  "tool": "pt"
}
```

Creating Custom Feedback Logging Fields

By default, the thumbs-up/thumbs-down feedback dialog offers a text box for an optional user comment. To capture more data about the user experience, you can configure Knowledge Assistant feedback dialog box with additional fields via the `logging_config.yaml` file described in [Configuring System and Feedback Logging](#).

Custom feedback fields are added in two steps:

1. Define new custom fields in a `custom_fields` section.
2. Reference the new custom fields in the `fields` list of fields in the `logging_config.yaml` file to include in logging entries.

The default feedback field definitions are as follows:

```
# logging_config.yaml
custom_fields:
  - name: feedback_reason
    label: Why did you dislike the answer?
    type: radio
    values:
      - incorrect: Incorrect answer
      - irrelevant: Irrelevant additional information
      - missing_info: Missing important information
    required: false
    category:
      - dislike
  - name: feedback_was_ka_faster
    label: Was Knowledge Assistant faster than other search methods?
    type: radio
```

```
values:
  - yes: Yes
  - no: No
required: false
category:
  - like
```

Multiple custom fields can be defined and included.

The following table describes the configuration parameters available for custom feedback fields defined in `custom_fields`:

Parameter	Description	Required?
<code>name</code>	Field name, consisting only of standard field characters - letters, numbers, and underscores, no leading number.	Yes
<code>label</code>	Human-readable description shown in the UI	No (default is no label)
<code>type</code>	Specifies the type of UI element. Valid values are: • <code>int</code> • <code>float</code> • <code>text</code> • <code>dropdown</code> • <code>radio</code>	Yes
<code>min, max</code>	Specifies numeric value limits. Valid only for the <code>int</code> and <code>float</code> types.	No (default is no limit)
<code>values</code>	List of values to show in UI element, as a YAML dictionary of key/value pairs. The values are selectable in the interface, and the corresponding keys are stored in the logging entry. Valid only for the <code>dropdown</code> and <code>radio</code> types.	Yes (for applicable types)
<code>required</code>	Specifies whether the user must enter a value for the field or not.	Yes
<code>category</code>	Relevant feedback type(s). Valid values are <code>like</code> and <code>dislike</code> .	Yes

6

The `copilot_admin` Administration Utility

The `copilot_admin` utility is a command-line administration tool for Synopsys.ai Copilot that provides comprehensive management capabilities for databases, content, models, and logging commands. This command-line tool allows administrators to initialize, configure, test, and maintain Synopsys.ai Copilot deployments.

Introduction

The `copilot_admin` utility is located in the `admin/bin/` directory of the Synopsys.ai Copilot installation directory:

```
$SNPSAI_COPILOT_HOME/admin/bin/copilot_admin
```

The `copilot_admin` command-line syntax consists of a command keyword, followed by a subcommand keyword, followed by any options specific to that command/subcommand:

```
copilot_admin command subcommand [-options...]
```

The `copilot_admin` functionality is organized into the four top-level command categories:

- `db` - commands for managing PostgreSQL databases and roles
- `docs` - commands for managing vector store document data
- `logs` - commands for managing conversation history logging data
- `model` - commands for managing AI models

Database Commands (db)

The `db` command category provides comprehensive database management functionality for both the `copilot_docs` and `copilot_logs` databases.

db show

Shows the current database configuration in YAML format.

Syntax

```
copilot_admin db show  
[--encrypt]
```

Arguments

--encrypt

Encrypts the values for the following configuration fields: `hostname`, `username`, `password`, `database`

Description

The `db show` command displays the database configurations for both the `copilot_docs` and `copilot_logs` databases in YAML format. It also shows the values of the following environment variables, if set:

```
PGUSER (superuser login)  
PGPASSWORD (superuser password)  
COPILOT_ADMIN_PGUSER (admin login)  
COPILOT_ADMIN_PGPASSWORD (admin password)
```

Comments and formatting from the original YAML endpoint configuration file are not preserved. If you use the `--encrypt` option, you will likely want to copy the encrypted settings back to your actual endpoint configuration file.

Already-encrypted values are shown as-is; they are not decrypted or re-encrypted.

db init

Initializes Copilot databases, roles, and required extensions.

Syntax

```
copilot_admin db init  
[--type all | docs | logs]
```

Arguments

--type `all` | `docs` | `logs`

Database type to operate on. Valid values are `all` (the default), `docs`, and `logs`.

Description

The `db init` command does the following:

- Creates the `copilot_admin` and `copilot_user` PostgreSQL roles
- Creates the `copilot_docs` and/or `copilot_logs` databases
- Installs the `pgvector` extension in the `copilot_docs` database

- Creates conversation history tables in the `copilot_logs` database
- Assigns database and table ownership to the `copilot_admin` role
- Grants database and table privileges to the `copilot_user` role

This command requires the following environment variables:

```
PGUSER (superuser login)
PGPASSWORD (superuser password)
COPILOT_ADMIN_PGUSER (admin login)
COPILOT_ADMIN_PGPASSWORD (admin password)
```

The `db init` command is an *idempotent* database command. This means it can be safely run multiple times without unintended side effects or changes to the state. It also means that the command can be run on existing databases with no data loss. Updates to privileges or ownership are applied incrementally as needed.

db test

Tests database connectivity, functionality, and latency.

Syntax

```
copilot_admin db test
  [--type all | docs | logs]
```

Arguments

`--type all | docs | logs`

Database type to operate on. Valid values are `all` (the default), `docs`, and `logs`.

Description

The `db test` command verifies database connections, tests basic commands, and measures connection latency to ensure the databases are functioning correctly.

The functional tests verify that:

- The `copilot_admin` role can create, read, write, and delete schemas and tables in the `copilot_docs` and `copilot_logs` databases
- The `copilot_user` role can read in the `copilot_docs` database
- The `copilot_user` role can read, write, and delete in the `copilot_logs` database

The latency tests measure the per-command performance for different combinations of SQL commands performed in transactions:

```
[INFO] ... Measuring connection latency to 'docs' database...
[INFO] ... SUCCESS: Completed (5 transactions) * (1 commands) in
0.175 seconds per command.
```

```
[INFO] ... SUCCESS: Completed (1 transactions) * (5 commands) in  
0.078 seconds per command.  
  
[INFO] ... Measuring connection latency to 'logs' database...  
[INFO] ... SUCCESS: Completed (5 transactions) * (1 commands) in  
0.173 seconds per command.  
[INFO] ... SUCCESS: Completed (1 transactions) * (5 commands) in  
0.076 seconds per command.
```

The latency test requests sent by this command are small and intended to measure only the round-trip network latency. They are not intended to measure the computational performance of the database.

This command requires the following environment variables:

```
COPILOT_ADMIN_PGUSER (admin login)  
COPILOT_ADMIN_PGPASSWORD (admin password)
```

db destroy

Destroys Copilot databases and roles.

Syntax

```
copilot_admin db destroy  
[--type all | docs | logs]
```

Arguments

```
--type all | docs | logs
```

Database type to operate on. Valid values are **all** (the default), **docs**, and **logs**.

Description

The **db destroy** command permanently deletes databases and roles. Use with extreme caution; this command cannot be undone.

This command requires the following environment variables:

```
PGUSER (superuser login)  
PGPASSWORD (superuser password)  
COPILOT_ADMIN_PGUSER (admin login)
```

Content Commands (docs)

The **docs** command category manages the document content stored in the **copilot_docs** vector store database.

docs list

Lists available database schemas.

Syntax

```
copilot_admin docs list  
[schema_names...]
```

Arguments

schema_names...

Names of specific schemas to list. The default is to show all schemas.

Glob-style wildcards (`?` and `*`) are supported. Patterns should be quoted to avoid being interpreted as file name patterns by the Linux shell.

Description

The `docs list` command displays the list of schemas in the database, formatted as one schema name per line.

This command requires the following environment variables:

```
COPILOT_ADMIN_PGUSER (admin login)  
COPILOT_ADMIN_PGPASSWORD (admin password)
```

docs dump

Exports database schemas to files for backup or propagation to other databases.

Syntax

```
admin docs dump  
schema_names...
```

Arguments

schema_names...

One or more names of schemas to dump.

Glob-style wildcards (`?` and `*`) are supported. Patterns should be quoted to avoid being interpreted as file name patterns by the Linux shell.

Description

The `docs dump` command writes one or more database schemas to disk. Each schema is written to an `*.sch` file named after the schema. The original schema name is stored in the file so that the file can be renamed as needed (such as to add a timestamp).

This command requires the following environment variables:

```
COPILOT_ADMIN_PGUSER (admin login)
COPILOT_ADMIN_PGPASSWORD (admin password)
```

docs restore

Restores database schemas from previously created *.sch backup files.

Syntax

```
copilot_admin docs restore
    file_names...
```

Arguments

file_names...

One or more file names of schema files to restore.

Glob-style wildcards (? and *) can typically be used, as they are detected and expanded by the Linux shell.

Description

The `docs restore` command restores schemas and their data from *.sch backup files created by the `docs dump` command. This command overwrites any existing schemas with the same names.

This command requires the following environment variables:

```
COPILOT_ADMIN_PGUSER (admin login)
COPILOT_ADMIN_PGPASSWORD (admin password)
```

docs delete

Permanently deletes database schemas.

Syntax

```
copilot_admin docs delete
    schema_names...
```

Arguments

schema_names...

One or more names of schemas to delete.

Glob-style wildcards (? and *) are supported. Patterns should be quoted to avoid being interpreted as file name patterns by the Linux shell.

Description

The `docs delete` command permanently deletes database schemas and all their content. Use with extreme caution; this command cannot be undone.

This command requires the following environment variables:

```
COPILOT_ADMIN_PGUSER (admin login)
COPILOT_ADMIN_PGPASSWORD (admin password)
```

Log Commands (logs)

The `logs` command category manages conversation logging data stored in the `copilot_logs` database.

logs dump

Exports conversation logging data to a file for analysis.

Syntax

```
copilot_admin logs dump
  --output_file file_name
  [--format json | csv]
  [--tools tool_keyword...]
  [--users user_name...]
```

Arguments

`--output_file file_name`

Output file to create.

A special value of `-` writes the output to STDOUT. This behavior is consistent with many Linux command-line utilities. It allows the output to be piped to subsequent commands for additional processing.

`--format json | csv`

Specifies the output format. Valid values are `json` (the default) and `csv`.

To maintain compatibility with Microsoft Excel, field values in CSV files are truncated to 32750 characters.

`-tools tool_keyword...`

Filters the output to activity for one or more specific tools. The default is to include activity for all tools.

```
--users user_name...
```

Filters the output to activity for one or more specific users. The default is to include activity for all users.

Description

The `logs dump` command extracts and writes conversation logging data to disk files. This data can be used for analysis, debugging, and creating reports.

This command requires the following environment variables:

```
COPILOT_ADMIN_PGUSER (admin login)  
COPILOT_ADMIN_PGPASSWORD (admin password)
```

logs import_json

Imports JSON-based Q&A metadata into the Postgres database.

Syntax

```
copilot_admin logs import_json
```

Description

The `logs import_json` command imports JSON-based Q&A metadata into the Postgres database.

In the X-2025.09-1 release, the following Q&A metadata fields were added to the Postgres database:

- `system_user_agent`
- `system_llm_canonical_name`
- `system_llm_config_name`
- `system_response_time`
- `system_token_tx`
- `system_token_rx`
- `system_command`
- `system_command_inferred`
- `scrutinized_question`
- `scrutinized_question_notes`
- `scrutinized_answer`

- `scrutinized_answer_notes`
- `wa_tcl_fixing`

However, Q&As created in Postgres before that release have empty values for those fields.

To populate those fields, the `logs import_json` command does the following:

- Locates the JSON logging directory using the `copilot_log_dir` application setting
- Reads in the JSON Q&A logging files
- Updates the matching Postgres Q&As to populate the fields above

This command requires the following environment variables:

```
COPILOT_ADMIN_PGUSER (admin login)
COPILOT_ADMIN_PGPASSWORD (admin password)
```

logs upgrade

Upgrades the logging database to the latest database schema version.

Syntax

```
copilot_admin logs upgrade
```

Description

The `logs upgrade` command updates the `copilot_logs` database to the latest database schema version.

This command requires the following environment variables:

```
COPILOT_ADMIN_PGUSER (admin login)
COPILOT_ADMIN_PGPASSWORD (admin password)
```

logs downgrade

Downgrades the logging database to a prior database schema version.

Syntax

```
copilot_admin logs downgrade
--revision revision
```

Arguments

`--revision`

Specifies the revision value to revert the database schema to. This value is required.

Description

The `logs downgrade` command updates the `copilot_logs` database to the prior database schema version specified by the `--revision` option.

Caution:

When downgrading a database schema, a loss of data can occur (such as to delete columns that are not part of an earlier schema).

This command requires the following environment variables:

```
COPILOT_ADMIN_PGUSER (admin login)
COPILOT_ADMIN_PGPASSWORD (admin password)
```

Model Commands (model)

The `model` command category provides functionality for testing and configuring the AI models used by Synopsys.ai Copilot.

model show

Shows current and available model configurations in YAML format.

Syntax

```
copilot_admin model show
  [--encrypt]
  [--all]
```

Arguments

`--encrypt`

Encrypts the values for the following configuration fields: `endpoint`, `api_key`

`--all`

Shows all model configurations. By default, only the currently selected model configurations are shown.

Description

The `model show` command shows the configuration for all model types (embedding, rerank, and LLM models) in YAML format.

Comments and formatting from the original YAML file are not preserved. If you use the `--encrypt` option, you will likely want to copy the encrypted settings back to your actual endpoint configuration file.

Already-encrypted values are shown as-is; they are not decrypted or re-encrypted.

model test

Tests embedding, rerank, and LLM models by sending test requests.

Syntax

```
copilot_admin model test  
[--all]
```

Arguments

`-all`

Tests all available models. The default is to test only the currently selected models.

Description

The `model test` command sends test requests to the embedding, rerank, and LLM models to verify their command and measure their latency.

After the tests complete, the command returns an exit status code as follows:

- If all tests pass, the command returns a status code of `0` (success).
- If any tests fail, the command returns a status code of `1` (failure).

This allows the command to be used for programmatic uptime monitoring of the embedding, rerank, and text generation models.

The requests sent by this command are small and intended to measure only the round-trip network latency. They are not intended to measure the computational performance of the models.

Examples

The following example shows the currently selected models being tested:

```
$ $SNPSAI_COPILOT_HOME/admin/bin/copilot_admin model test  
...  
[INFO] ... SUCCESS: Completed 'embedding' request in 0.565 seconds  
to 'my-embedding-model-1'.  
[INFO] ... SUCCESS: Completed 'rerank' request in 0.656 seconds  
to 'my-rerank-model-2'.  
[INFO] ... SUCCESS: Completed 'llm' request in 0.775 seconds  
to 'my-llm-model-1'.
```

The following example shows all available models being tested:

```
$ $SNPSAI_COPILOT_HOME/admin/bin/copilot_admin model test --all
...
[INFO] ... SUCCESS: Completed 'embedding' request in 0.565 seconds
to 'my-embedding-model-1'.
[INFO] ... SUCCESS: Completed 'embedding' request in 0.483 seconds
to 'my-embedding-model-2'.
[INFO] ... SUCCESS: Completed 'rerank' request in 0.656 seconds
to 'my-rerank-model-1'.
[INFO] ... SUCCESS: Completed 'rerank' request in 0.548 seconds
to 'my-rerank-model-2'.
[INFO] ... SUCCESS: Completed 'llm' request in 0.775 seconds
to 'my-llm-model-1'.
[INFO] ... SUCCESS: Completed 'llm' request in 0.824 seconds
to 'my-llm-model-2'.
```

model ask

Sends a query to Knowledge Assistant for end-to-end testing of the RAG pipeline.

Syntax

```
copilot_admin model ask
  --tool tool_keyword
  [--release tool_release]
  --query user_query
```

Arguments

`--tool tool_keyword`

Tool keyword to use for the query.

`--release tool_release`

Tool release to use for the query. By default, the latest available release is used.

`--query user_query`

The query to submit to Knowledge Assistant.

Description

The `model ask` command sends a query to Knowledge Assistant to perform an end-to-end test of the database and model functionality in the RAG pipeline.

This command is intended only to test functionality; the answer is not returned.

model list

Lists all available models.

Syntax

```
copilot_admin model list
```

Description

The `model list` command lists all available models (embedding, rerank, and LLM models) in table format.

Examples

The following example demonstrates the usage:

```
$ $SNPSAI_COPILOT_HOME/admin/bin/copilot_admin model list
...
```

model_type	config_name	model_canonical_name
embedding	my-embedding-model-1	embedding-model-1
embedding	my-embedding-model-2	embedding-model-2
rerank	my-rerank-model-1	rerank-model-1
rerank	my-rerank-model-2	rerank-model-2
llm	my-llm-model-1	llm-model-1
llm	my-llm-model-2	llm-model-2

A

Host-Tool-Based Invocation Information

The following table lists the tools that support invocation of Knowledge Assistant from within a tool session, along with their minimum required releases for it.

Table 6: Knowledge Assistant Invocation from a Host Tool

Tool	Minimum required release	Invocation method
Custom Compiler	W-2024.09-SP1-1	<code>tool_prompt> copilot</code>
Fusion Compiler	W-2024.09-SP3	<code>tool_prompt> copilot</code>
IC Validator	W-2024.09-SP3	<code>linux_prompt> icv -copilot</code>
PrimeSim Pro	X-2025.06	<code>linux_prompt> primesim -copilot</code>
PrimeSim SPICE	X-2025.06	<code>linux_prompt> primesim -copilot</code>
PrimeSim XA	X-2025.06	<code>linux_prompt> xa -copilot</code>
PrimeTime	W-2024.09-SP2	<code>tool_prompt> copilot</code>
TestMAX ATPG	X-2025.06	<code>tool_prompt> copilot</code>
VC Formal	W-2024.09-SP1	<code>tool_prompt> copilot</code> ¹
VC Low Power	W-2024.09-SP1	<code>tool_prompt> copilot</code>
VC SpyGlass	W-2024.09-SP1	<code>tool_prompt> copilot</code>
VCS	W-2024.09-SP2	<code>linux_prompt> vcs -copilot -full64</code>
Verdi	W-2024.09-SP2-1	GUI Help > Knowledge Assistant ²

Direct invocation of Knowledge Assistant (using the `snpsai_copilot` command) has no requirement on the host tool release. You can use direct invocation if Knowledge Assistant and host tool licenses are available, regardless of what release of the host tool is installed (because direct invocation is not performed through the host tool).

1. If the `SNPS_VCST_ENABLE_COPILOT` environment variable is set to 1 when VC Formal is invoked, a **Help > Copilot** menu item will also be available.
2. A `GenAI_Verdi_Beta` license is required to enable this menu item.

B

Supported Models

The following topics list the models supported by Synopsys.ai Copilot in this release:

- [Supported Embedding Models](#)
- [Supported Rerank Models](#)
- [Supported LLM Models](#)

Supported Embedding Models

The following table lists the embedding models supported by Synopsys.ai Copilot in this release. Legacy canonical names follow the preferred canonical names in parentheses.

Canonical embedding model name	Dimensions
amazon/titan-embed-text-v2.0 (amazon/Titan-text-embeddings-v2)	1024
baai/bge-large-en-v1.5 (BAAI/bge-large-en-v1.5)	1024
google/gemini-embedding-001 (gcp-vertex/gemini-embedding-001)	768
nomic/nomic-embed-text-v1.5 (nomic-ai/nomic-embed-text-v1.5)	768
openai/text-embedding-3-large (text-embedding-3-large-1024)	1024

Other embedding models *cannot* be used, as vector stores can only be semantically searched using the same model that generated the vectors.

Supported Rerank Models

The following table lists the rerank models supported by Synopsys.ai Copilot in this release.

Canonical rerank model name

`baai/bge-reranker-v2-m3`
`baai/bge-reranker-v2.5-gemma2-lightweight`
`cohere/rerank-v3.5`
`cohere/rerank-v4.0-fast`
`cohere/rerank-v4.0-pro`
`google/semantic-ranker-default-004`
`qwen/qwen3-reranker-4b`
`qwen/qwen3-reranker-8b`

In addition, any rerank model supported by SCS AI Gateway can be used.

The following rerank models are deprecated and should not be used:

- `baai/bge-reranker-base`
- `baai/bge-reranker-large`

Supported LLM Models

The following table lists the LLM models supported by Synopsys.ai Copilot in this release. Legacy canonical names follow the preferred canonical names in parentheses.

Canonical LLM model name

`anthropic/claude-3.7-sonnet-20250219`
`anthropic/claude-opus-4.5-20251101`
`anthropic/claude-sonnet-4.5-20250929`
`google/gemini-2.5-flash`
`google/gemini-2.5-pro`
`openai/gpt-4.1 (gpt-4.1)`
`openai/gpt-4o (gpt-4o)`
`openai/gpt-oss-120B`

Canonical LLM model name

`qwen/qwen3-coder-480b-a35b-instruct`

The following LLM model is deprecated and should not be used:

- `meta/llama-3.3-70b-instruct`